

1. INSERT (Meter datos)

Lo básico es meter filas. El PDF nos enseña dos formas, pero tú quédate con la **segura** (especificando columnas).

- **Sintaxis Segura:**

SQL

```
INSERT INTO planetas (nombre, distancia_tierra)
VALUES ('Mercurio', 77000000);
```

Nota: Fíjate que en el PDF la tabla tiene un id auto-incremental. Al hacer el insert, **NO** mencionamos la columna id y MySQL le pone el número solo.

- **Truco para el examen (Insert Múltiple):**

Puedes meter muchos de golpe separándolos por comas (ahorras tiempo):

SQL

```
INSERT INTO planetas (nombre, distancia_tierra) VALUES
('Venus', 40000000),
('Marte', 225000000);
```

2. UPDATE (Modificar datos)

Aquí está el peligro. Sirve para cambiar datos que ya existen.

- **Sintaxis:**

SQL

```
UPDATE nombre_tabla SET columna = nuevo_valor WHERE condicion;
```

- **ALERTA DE EXAMEN (El Apocalipsis):**

Si se te olvida el WHERE, **modificas TODAS las filas de la tabla.**

- *Ejemplo del PDF:* Si quieres sumar 100.000km a *todos* los planetas, ahí sí lo haces sin WHERE .
- *Pero si quieres corregir solo a Venus:* WHERE nombre = 'Venus' .

3. DELETE vs TRUNCATE (Borrar datos)

El PDF hace mucho énfasis en esta diferencia, es pregunta de teoría fija .

Comando	Descripción	¿Se puede deshacer?	¿Respeto FK?
DELETE	Borra filas una a una.	Sí (con	Sí. Comprueba si hay hijos

Comando	Descripción	¿Se puede deshacer?	¿Respetar FK?
	Admite WHERE.	transacciones).	(ON DELETE CASCADE/RESTRICT).
TRUNCATE	Vacia la tabla entera de golpe. Es muy rápido.	No.	No. Falla si hay claves ajenas apuntando a ella.

- **Tu regla:** Usa siempre DELETE a menos que quieras reiniciar la tabla a cero (y resetear el contador del auto_increment).

Solución Actividad 1: Actores y Personajes (Pág. 6-7)

Este ejercicio es buenísimo porque mezcla DML con las **relaciones (Foreign Keys)** y el **Cascade**.

Escenario:

1. Tabla actores (Padre).
2. Tabla personajes (Hija, apunta a actor).
3. FK definida con ON DELETE CASCADE.

Aquí tienes el código paso a paso:

A. Preparación (Insertar datos)

SQL

-- Insertamos actores (el id es auto_increment, no lo pongo)

INSERT INTO actores (nombre, fecha_nacimiento) VALUES

('Penélope Cruz', '1974-04-28'),

('Javier Bardem', '1969-03-01'),

('Antonio Banderas', '1960-08-10'),

('Carmen Maura', '1945-09-15'),

('Blanca Suárez', '1988-11-21'), -- Ojo a esta fecha errónea

('Emilio Aragón', '1959-04-16');

-- Insertamos personajes (asumiendo IDs: Penélope=1, Antonio=3)

INSERT INTO personajes (nombre, codigo_actor) VALUES

('Raimunda', 1), -- Penélope

('Anton Chigurh', 3); -- Antonio (Está mal a propósito para luego corregirlo)

B. Modificaciones (Updates)

1. **Corregir fecha de Blanca Suárez:**

SQL

UPDATE actores

```
SET fecha_nacimiento = '1988-10-21'
```

```
WHERE nombre = 'Blanca Suárez';
```

2. **Corregir el actor de Anton Chigurh:** (*Anton Chigurh lo hizo Javier Bardem, ID 2, no Banderas*)

SQL

```
UPDATE personajes
```

```
SET codigo_actor = 2
```

```
WHERE nombre = 'Anton Chigurh';
```

C. Borrados y Cascada (Deletes)

1. **Borrar a Emilio Aragón:**

SQL

```
DELETE FROM actores WHERE nombre = 'Emilio Aragón';
```

2. **La Pregunta del Millón:** *"Borrar a Penélope Cruz. ¿Por qué no da error si hay una referencia suya en la tabla personajes?"*

SQL

```
DELETE FROM actores WHERE nombre = 'Penélope Cruz';
```

Respuesta para el examen: No da error porque al crear la tabla personajes definimos la clave foránea con **ON DELETE CASCADE**. Al borrar a Penélope (padre), MySQL automáticamente borró a "Raimunda" (hija). Si hubiera sido **RESTRICT**, habría dado error.

3. **Limpieza total:**

SQL

```
-- Opción lenta
```

```
DELETE FROM personajes;
```

```
DELETE FROM actores;
```

```
-- Opción rápida (pero cuidado con las FKs, truncate actores fallaría si personajes tiene datos)
```

```
TRUNCATE personajes;
```

```
-- Ahora que personajes está vacía, sí deja truncar actores
```

```
SET FOREIGN_KEY_CHECKS = 0; -- Truco pro para forzar truncate
```

```
TRUNCATE actores;
```

```
SET FOREIGN_KEY_CHECKS = 1;
```

Resumen para tu chuleta (UT 3.3)

1. **INSERT INTO tabla (col1, col2) VALUES (val1, val2);** -> Siempre especifica columnas.
2. **UPDATE** -> ¡No olvides el **WHERE** o la lías parda!
3. **DELETE vs TRUNCATE** -> Delete borra selectivamente y dispara

triggers/cascadas. Truncate reinicia la tabla a lo bruto.

4. **Fechas** -> Siempre entre comillas y formato 'AAAA-MM-DD'.